Closest Pair of Points: Divide and Conquer Visualizer

Cheolwon Park (Student ID: 2024270616)
Korea University
scott9020@korea.ac.kr

1 Web Application and Algorithm Implementation

A closest pair of points visualizer was built using Flask (Python backend) and HTML/CSS/JavaScript with Canvas (frontend). The application generates n random 2D points on an 800×600 canvas and finds the closest pair using two approaches, comparing their execution times:

Algorithm	Complexity	Method
Brute Force	$O(n^2)$	Check all n(n−1)/2 pairs, track minimum distance
Divide & Conquer	$O(n \log^2 n)$	Split by x-coordinate, recurse on halves, check strip

The user enters the number of points and clicks "Generate Random" to scatter points on the canvas. Clicking "Find Closest Pair" runs both algorithms on the server, highlights the closest pair in red with a connecting line, and displays a comparison table with execution times and the speedup ratio.

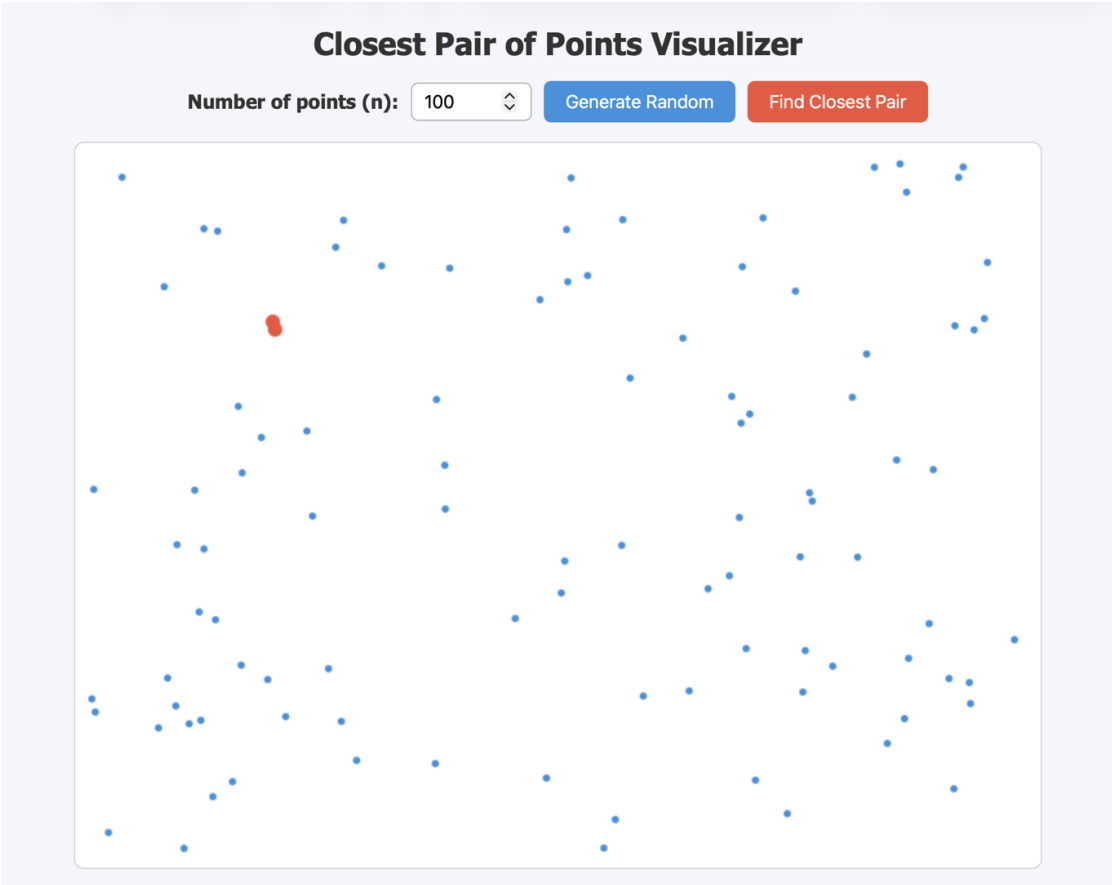

Figure 1. Closest pair highlighted in red on the 2D canvas (n=100)

2 Performance Comparison (N = 100, 1000, 5000)

Each algorithm was tested on randomly generated 2D points. All tests used `time.perf_counter()` for precise timing.

Number of points	Brute Force	Divide & Conquer	Speedup
100	2.1366	0.2940	7.27×
1,000	124.1273	1.3770	90.15×
5,000	2,108.5178	7.4540	282.87×

Table 1. Performance comparison across point counts

As n grows, the speedup increases dramatically. At n = 5,000, the divide and conquer approach is about 283 times faster than brute force, clearly reflecting the gap between $O(n^2)$ and $O(n \log^2 n)$.

3 Analysis

3.1 How does Divide and Conquer reduce $O(n^2)$ to $O(n \log^2 n)$?

The brute force method examines every pair of points, giving $n(n-1)/2$ comparisons and $O(n^2)$ time. The divide and conquer algorithm improves on this through three steps:

1. **Divide:** Sort all points by x-coordinate, then split them into two equal halves at the median. Each half contains n/2 points.
2. **Conquer:** Recursively find the closest pair in the left half and the right half. Let d be the smaller of the two distances found.
3. **Combine:** Build a "strip" of points within distance d of the dividing line, sort them by y-coordinate, and check cross-half pairs inside the strip. This combine step takes $O(n \log n)$ time due to sorting.

The recurrence is $T(n) = 2T(n/2) + O(n \log n)$, which solves to $O(n \log^2 n)$ by the Master Theorem. Empirically, brute force time grew ~987× from n=100 to n=5,000 (close to $50^2/2 = 1{,}250$), while D&C grew only ~25×, consistent with the $n \log^2 n$ prediction.

3.2 Why is checking only the strip sufficient?

If the true closest pair crosses the dividing line, both points must lie within distance d of that line — otherwise their horizontal distance alone would exceed d, making them farther apart than the best pair already found within one half. Furthermore, for each point in the strip, at most 7 other points can lie within a $d \times 2d$ rectangle around it (proven by a packing argument: dividing the rectangle into 8 cells of side $d/2$, each containing at most one point). This geometric bound guarantees the strip check examines $O(n)$ pairs total, making it both correct and efficient.